# AI Algorithms in Society — SC3098
# Group Design Project
# Lab #3 — Explainability & Human Oversight

**Dr Zhang Jiehuang**

College of Computing and Data Science

Nanyang Technological University

email: *jiehuang.zhang@ntu.edu.sg*

Tuesday 8.30 – 10.20am · Software Project Lab · Week 4

# Today's Session

| | Block | What you will do |
|---|---|---|
| 1 | **Why it matters** | Trust, suitability and accountability in financial advice — and the regulation that makes all three non-negotiable rather than nice to have. |
| 2 | **Explain & attribute** | Cite the sources behind an answer (C.1) and explain each recommendation in the client's own terms (C.2), including which explanation technique fits which case. |
| 3 | **Confidence & escalation** | A calibrated confidence signal, an honest abstain path, and human override on high-stakes decisions (C.3, C.4). |
| 4 | **Accountability** | A queryable audit log and provenance trail (C.5), the four ways this most often goes wrong, and booking your checklist sign-off. |

*Each block ends in something demonstrable. Checklist items C.1 – C.5 are the assessment, not extra work.*

PART 1

# Why Explainability and Human Oversight?

A robo-advisor gives financial advice — the bar for trust is set by the regulator, not by us.

# Why a Robo-Advisor Must Explain Itself

**Suitability**

- Advice must fit the client's risk profile, horizon and goals — and the firm must be able to demonstrate, after the fact, that it did. Suitability is a duty owed to the client, not a property of the model.

**Accountability**

- When advice turns out to be wrong, someone must be able to trace how the decision was reached and answer for it. "The model decided" is not an answer a regulator accepts.

**User trust**

- People act on advice they understand. Unexplained output gets ignored — or, worse, followed blindly by someone with no way to tell a good recommendation from a bad one.

**High stakes**

- This is money, not text. Errors compound over years, several are effectively irreversible, and the client often finds out long after the damage is done.

*None of these four are satisfied by a model that is merely accurate. Accuracy is what Lab #2 was for.*

# What Regulators Actually Require

| Framework | What it requires of you | Lands on |
|---|---|---|
| **MAS FEAT (Singapore)** | Fairness, Ethics, Accountability and Transparency — MAS' principles for AI and data analytics in financial services. Transparency: explain AI-driven decisions in language the customer can follow. Accountability: named internal ownership of every model. | **C.1 C.2 C.4** |
| **Suitability duty (robo-advisory guidance)** | Digital advisers must collect enough client information, ensure the recommendation is suitable for that client, and retain records that evidence both. "We cannot reconstruct it" is not a defence. | **C.2 C.5** |
| **EU AI Act (high-risk uses)** | Creditworthiness assessment and insurance pricing are named high-risk. Automatic logging, human oversight, and information for the deployer are obligations rather than features. | **C.3 C.4 C.5** |

*You are not building nice-to-haves. C.1 – C.5 are the mechanism by which a robo-advisor evidences that it met these duties.*

# Five Layers, Five Checklist Items

| Item | Layer | What it means in your build |
|---|---|---|
| C.1 | **Source attribution** | Show which retrieved passages the answer was built from, with document, section and date. |
| C.2 | **Local explanation** | Say why this recommendation, for this client, in terms of the inputs they gave. |
| C.3 | **Confidence & abstain** | Say how sure the system is — and refuse to answer when the evidence is thin. |
| C.4 | **Human-in-the-loop** | Pause high-stakes decisions for a reviewer who can approve, edit or override. |
| C.5 | **Audit log** | Record enough that the decision can be reconstructed and explained months later. |

*Build them in this order — each one feeds the next. You will not finish all five today; scaffold them, and book sign-off the moment one works.*

PART 2

# Explain and Attribute

C.1 Source attribution

C.2 Local explanation of recommendations

# C.1 Source Attribution and Traceability

*Every substantive answer should surface the retrieved passages it was built from, with provenance the user can inspect — document title, section and date.*

**Carry metadata**

- Attach source, section and date to every chunk at ingest time — never at answer time, when the link has already been lost.

**Return top-k sources**

- Pass the retrieved passages into the response object, not just the generated answer string.

**Make it inspectable**

- The interface lets a user expand a citation to the exact source text that was retrieved.

**Common failure**

- A document name on its own is not traceability — the passage itself has to be reachable.

*Sign-off: expand a citation to the exact source passage, for three different queries.*

# Hands-On #1 — Wire Citations Into One Response (16 min)

## DO NOW

1. Pick one advisor response path — for example, “why this fund?”
2. Thread the retrieved passages and their metadata through to the frontend
3. Render an expandable “Sources” panel underneath the answer
4. Verify that the cited text really is what the retriever returned

## FACILITATION

- Supervisors circulate — check that metadata survives retrieval
- Common bug: the chunk text is kept but the source id is dropped upstream
- Stuck on the interface? A plain expandable list is enough for sign-off
- Debrief: one team demonstrates a citation traced back to its source

# C.2 Explaining Each Recommendation

*The client should see why this recommendation, expressed in terms of the inputs they themselves gave — not a generic description of how the model works.*

**Rationale**

- A short “why” tied to this client’s profile, horizon and stated constraints — two or three sentences, not a lecture on the model.

**Attribution**

- For scored or ranked outputs, show which features drove the score and in which direction each one pushed it.

**Evidence**

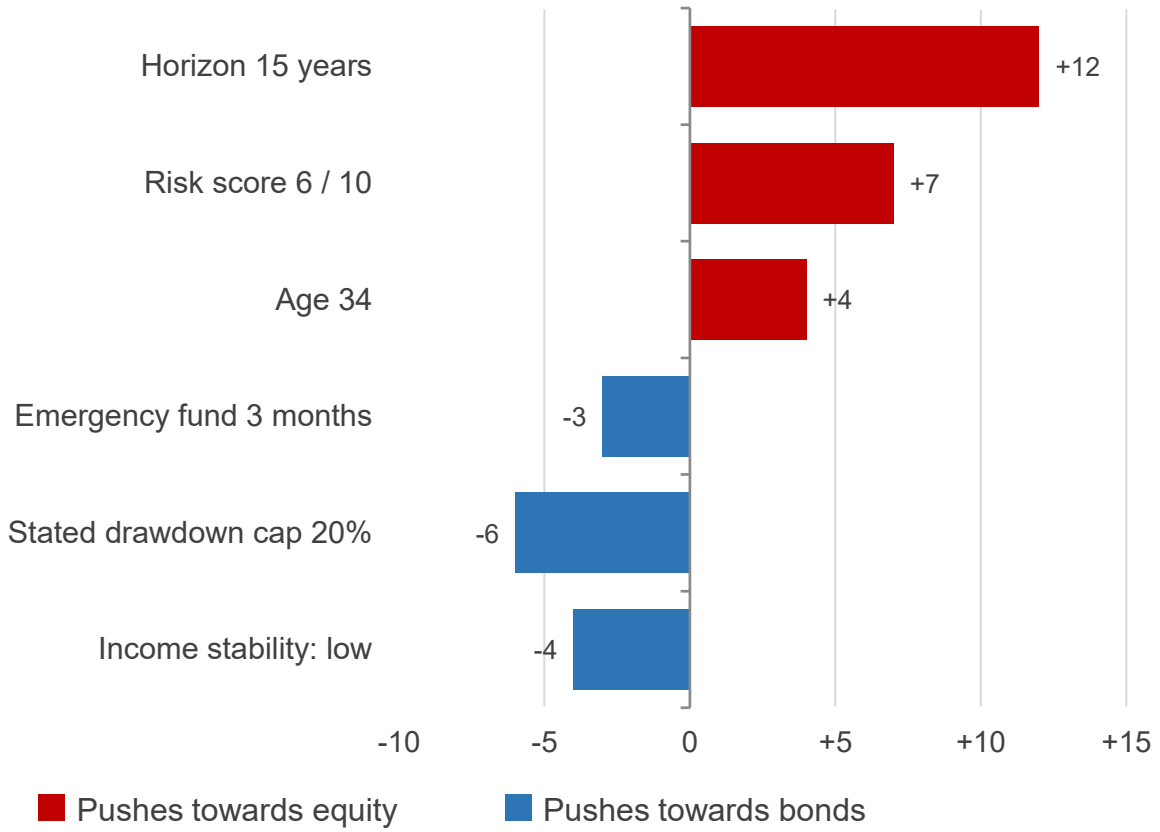
- Point at the specific inputs or retrieved passages behind the call, so the client can check them independently.

**Reactive**

- Change an input and the explanation must change with it. A static explanation is decoration, not evidence.

*Sign-off: the explanation sits beside the recommendation and updates correctly when an input changes.*

# Global, Local and Counterfactual Explanations




**Global — what the model does in general**

- Feature importance across the whole dataset. Answers "what drives our risk scores?" Right for model validation and governance; wrong for telling one client why they got this portfolio.

**Local — why this output, for this input**

- SHAP distributes a single prediction across its features, as in the chart. LIME fits a simple surrogate near one point. This is the form C.2 asks for.

**Counterfactual — what would have to change**

- "Had your horizon been 10 years rather than 3, this shifts to 70% equity." Usually the most actionable for the client — but only offer changes they could actually make.

*Local attribution for one client: contribution of each input to the 70% equity weight, in percentage points from a 60% base.*

# The Same Recommendation, Explained Two Ways

**Scenario:** Client is 34, moderate risk tolerance (6/10), 15-year horizon, S$60,000 to invest. The system recommends a 70/30 global equity-bond portfolio.

## NOT ACCEPTABLE

> "Based on our analysis, this portfolio is well-suited to your profile and offers strong long-term growth potential."

### Why it fails

- Names none of the inputs the client actually gave
- Unfalsifiable — the same sentence would fit any client
- Nothing to inspect, so nothing the client can challenge
- Cannot be reconstructed from the log six months later

## ACCEPTABLE

> "70% equity because your 15-year horizon and moderate risk score (6/10) place you in the growth band. 30% bonds caps expected drawdown near the 20% you said you could tolerate. Source: Allocation Policy v2.1, §3.2 (Mar 2026). Confidence: high (0.87)."

### Why it passes

- Names the specific client inputs it relied on
- Cites the governing policy section, with version and date
- States confidence, so the client can weigh it themselves
- Every element above is written into the audit entry

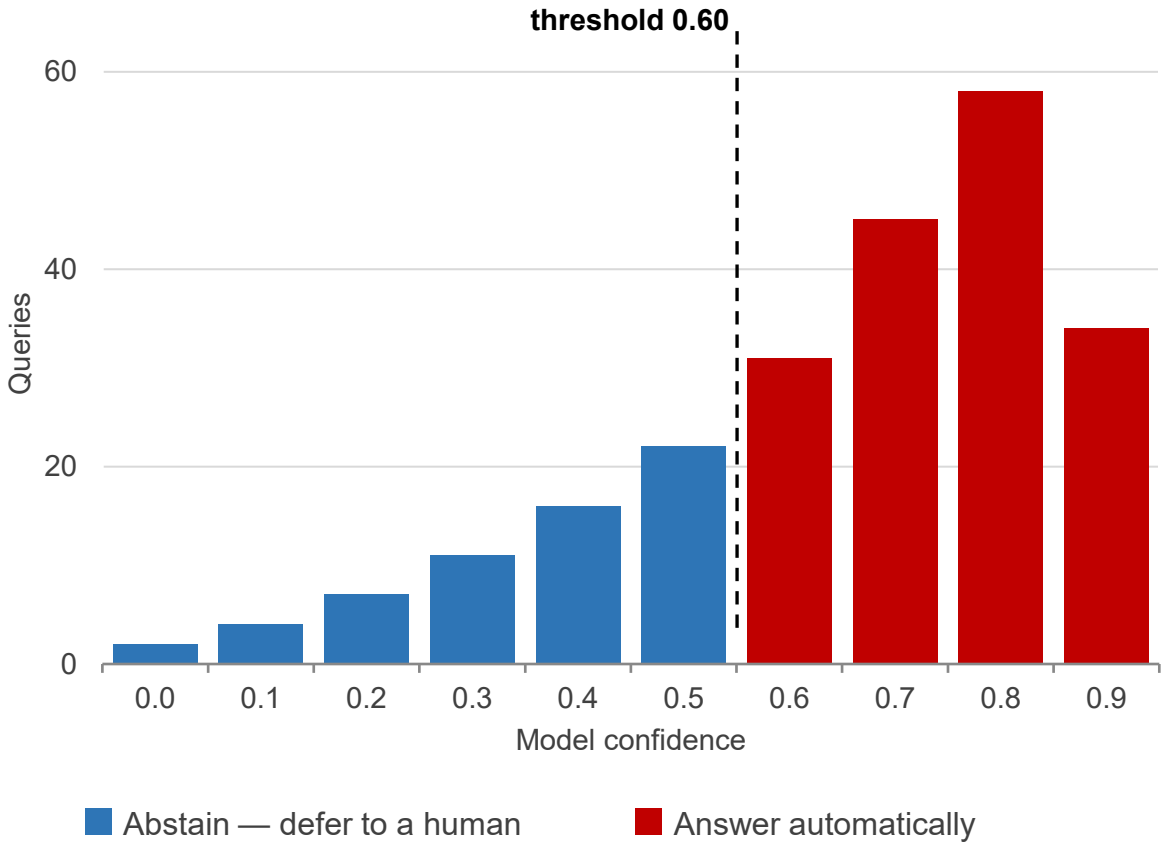
PART 3

# Confidence and Escalation

C.3 Calibrated confidence and abstain

C.4 Human-in-the-loop override

# C.3 Confidence, Uncertainty and Abstain




**Score it**

- Derive a confidence signal — retrieval score, self-consistency across samples, or agreement between models.

**Show it**

- Surface a clear indicator next to the answer, not a raw number buried in a log line. The client has to be able to act on it.

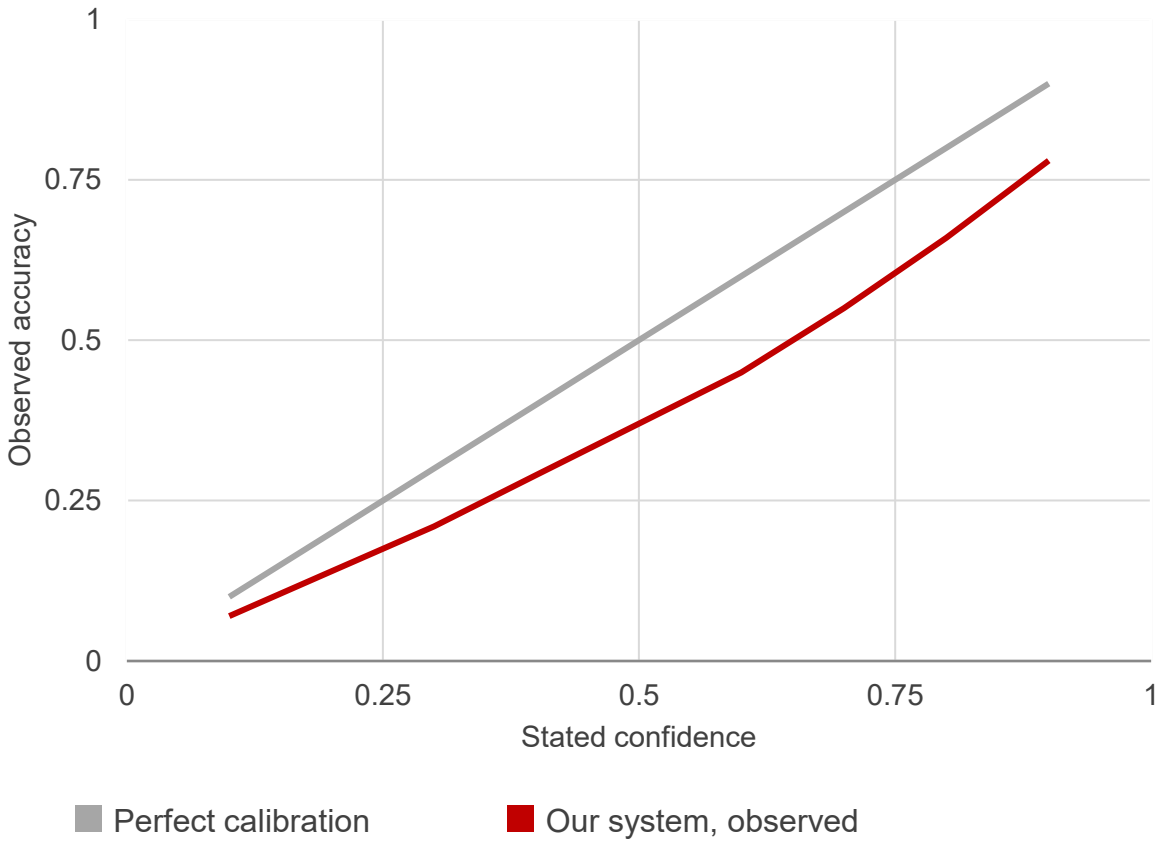
**Abstain**

- Below the threshold, say “I’m not sure” and route onward to a human. A confident guess is the worst available output.

**Sign-off: a low-confidence query trips a visible caution and abstains at a defined threshold.**

*One week of queries. At a 0.60 threshold roughly 27% are deferred rather than answered automatically.*

# Where a Confidence Number Comes From




- Retrieval score — cosine similarity of the top-k passages. Cheap, already in your pipeline, degrades when the corpus is thin.
- Self-consistency — sample the answer n times, treat disagreement as uncertainty. Reliable, costs n× inference.
- Model-reported — ask the LLM how confident it is. Convenient, and systematically overconfident.
- Ensemble or margin — distance from the decision boundary, or variance across an ensemble.

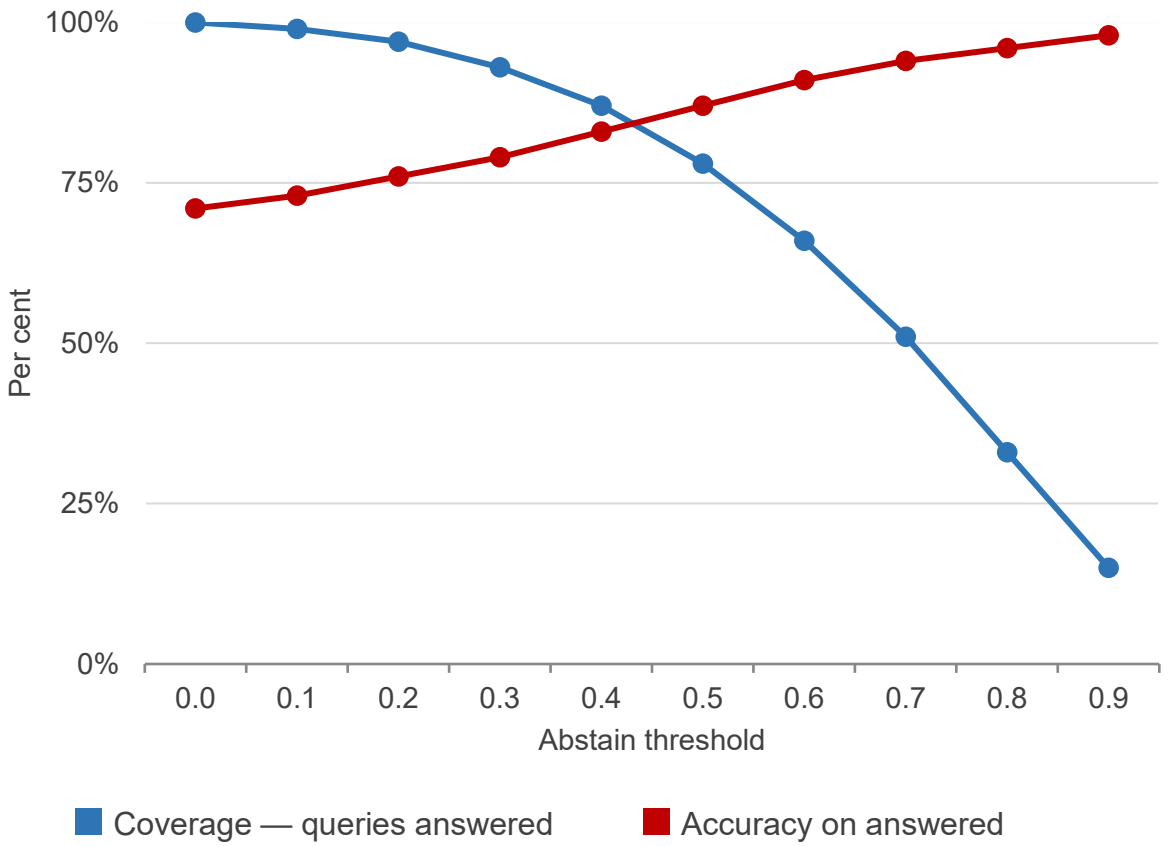
**Calibration is the point**

A confidence score is calibrated when, across every occasion the system says 80%, it is right about 80% of the time. The chart on the left is how you check it; Platt scaling or isotonic regression fixes it.

For this lab, a rough uncalibrated signal plus a working abstain path clears C.3.

*Reliability diagram. Below the diagonal is overconfident — the system claims more certainty than it earns.*

# Choosing the Threshold: the Abstain Trade-Off




**There is no free threshold**

- Every point on the red curve is bought with a point on the blue one. A system that abstains on nothing is not confident, it is untuned.

**Pick it from the cost of being wrong**

- For financial advice, a wrong recommendation costs far more than a deferral costs. That argues for a threshold to the right of where accuracy alone would put it.

**State the operating point**

- Your report should name the threshold you chose, the coverage it produces, and why that balance is defensible for this domain.

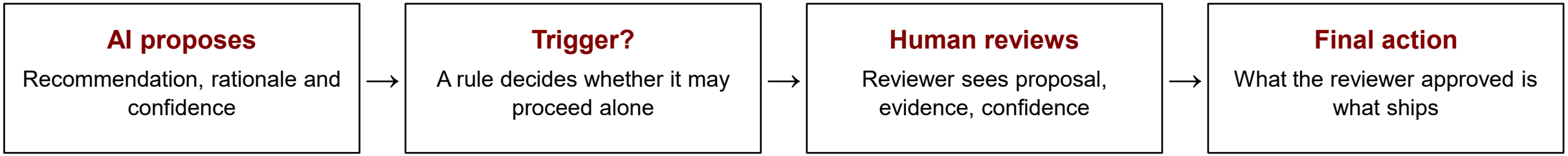
**A useful sanity check**

- If your abstain rate is 0%, your threshold is doing nothing. If it is above 50%, you have built a referral form rather than an advisor.

*Raising the abstain threshold buys accuracy on the queries you still answer — and pays for it in coverage.*

# C.4 Human-in-the-Loop Review and Override

*High-stakes or low-confidence decisions pause for a human. It is the reviewer's decision, not the raw model output, that ships.*




**Triggers:** a large allocation change · any use of leverage · confidence below threshold · a client-vulnerability flag

| The reviewer may | What that means | What must be recorded |
|---|---|---|
| **Approve** | The proposal ships unchanged. | The approval itself, against a named reviewer. |
| **Edit** | The reviewer amends the recommendation; the amended version is what the client sees. | The original, the edit, and who made it. |
| **Override** | The reviewer rejects it outright and substitutes their own call. | The substitution and the reason given. |

*Sign-off: run one override end-to-end — the human's edited decision is what the system finally acts on.*

# Hands-On #2 — Confidence, Abstain and Override (24 min)

## DO NOW

1. Attach a confidence signal to one recommendation path
2. Define a threshold and an abstain response for anything below it
3. Pick one high-stakes trigger and pause the flow for review
4. Stub approve / edit / override — and ship the human's choice, not the model's

## FACILITATION

- A hard-coded threshold is fine — the behaviour is what is assessed
- Check that the override actually replaces the AI output downstream
- Watch for: reviewer edits shown in the interface but ignored by the service
- Debrief: contrast one team's abstain path with its override path

*Leave the checkpoint slide up while they build, and call the times out loud.*

# Hands-On #2 — Checkpoints

*If a milestone slips, drop the polish and move on — everything after it depends on it.*

| At | Milestone | What "done" looks like |
|---|---|---|
| **+5 min** | **Signal exists** | One number, from any source, printed beside a recommendation. Do not tune it yet — just make it exist and flow through. |
| **+12 min** | **Abstain fires** | A hard-coded threshold produces a visibly different response for a deliberately weak query. Any threshold will do. |
| **+18 min** | **Trigger pauses** | One high-stakes condition halts the flow and surfaces a review prompt to the reviewer instead of an answer to the client. |
| **+24 min** | **Override ships** | The reviewer's edited text is what the client-facing response actually contains — not merely what the interface displays. |

*Supervisors: call these out loud. A team still tuning its confidence score at +12 min should hard-code the number and move to the abstain path.*

# Four Failure Modes to Check Before Sign-Off

| Failure mode | What actually happens | How to test for it |
|---|---|---|
| **The confident citation that isn't** | The model generates a plausible-looking source name that was never retrieved. | *Assert every citation id appears in the retrieval result before you render it.* |
| **The threshold that never fires** | Confidence is tuned upward until abstaining stops happening, because abstaining looks like failure in a demo. | *Find a query that should abstain, and confirm that it does.* |
| **The override that goes nowhere** | The reviewer edits the recommendation, the interface shows the edit, and the downstream service still sends the model's original. | *Assert on what actually ships, not on what the screen displays.* |
| **The log you cannot query** | Everything is written — into a free-text log line. Nothing can be reconstructed six months later. | *Pull one past decision by id, cold, in front of someone else.* |

PART 4

# Accountability and Sign-Off

C.5 Audit log and provenance

Booking your checklist verification

# C.5 Audit Log and Provenance

*One queryable, timestamped record per decision — enough that any past recommendation can be reconstructed and explained after the fact.*

| Field | What to store | Why it has to be there |
|---|---|---|
| `decision_id` | Unique id, timestamp, session reference | The handle everything else hangs off |
| `client_inputs` | Profile, risk score and constraints as submitted | Shows what the advice was actually based on |
| `retrieved_sources` | Document ids, sections, versions | Reproduces the evidence trail (C.1) |
| `model_output` | Recommendation, rationale, confidence | The proposal before any human touched it |
| `human_action` | Approve / edit / override, plus reviewer id | Who is accountable for what shipped (C.4) |
| `final_response` | What the client actually saw | The only thing they could have acted on |
| `versions` | Model, prompt and policy-document versions | Explains behaviour that changed over time |

*Sign-off: retrieve the full provenance trail for a past decision the supervisor picks at random.*

# Hands-On #3 — Log One Decision End-to-End (14 min)

**DO NOW**

1. Write one structured entry per recommendation
2. Include the inputs, output, confidence, sources and versions
3. Record any human override against that same entry
4. Prove you can query one past decision back out again

**FACILITATION**

- A JSON-lines file or a database table both qualify
- Reuse the sources object from Hands-On #1 (C.1)
- Reuse the override result from Hands-On #2 (C.4)
- Debrief: a supervisor names a query, the team pulls its trail

*Verify the read path, not the write path — plenty of teams write beautifully and cannot get anything back out.*

# Getting Signed Off: What to Show a Supervisor

| Checklist item | What to demonstrate | Typical |
|---|---|---|
| **C.1 Source attribution** | Expand a citation to its exact source text, for three different queries | **~5 min** |
| **C.2 Local explanation** | Change one client input and show the explanation change with it | **~5 min** |
| **C.3 Confidence & abstain** | A deliberately weak query trips a visible caution and abstains | **~5 min** |
| **C.4 Human-in-the-loop** | A full override — the reviewer's edit is what the client receives | **~10 min** |
| **C.5 Audit log** | Reconstruct a past decision that the supervisor picks | **~5 min** |

*Book a verification the moment an item works — do not wait until all five are finished. Slots fill up in Week 5.*

# Before Next Week

## REQUIRED

- Checklist C.1 – C.5, Explainability & Human Oversight. This is the document you are assessed against; read it as a specification, not as background.
- Tutorial #3 — Explainability & Human Oversight (Monday). Re-read the worked examples; they use the same scenario as the slide earlier in this deck.
- Assign an owner per checklist item within your team, and pair that split with the software and architecture work already in flight.
- Bring a laptop running the live system next week — sign-off is done against a running build, never from screenshots.

**NEXT WEEK — Lab #4: Development and clearing the checklist**

Finish C.1 – C.5, integrate them with the core system rather than leaving them as demos, and start booking supervisor sign-offs in earnest. Sign-off slots open in Week 5 — and they fill.

KEY TAKEAWAY

# Trustworthy Beats Clever

A robo-advisor that explains itself, states how sure it is, knows when to stop, and keeps a record it can be held to, is worth more than one that is merely accurate.

**Book your first checklist sign-off before you leave this session.**